

СИСТЕМНЫЙ ДИЗАЙН AI-ПЛАНИРОВЩИКА

Интеграция с Google Calendar и Apple Calendar

1.1 Функциональные и нефункциональные требования

Функциональные требования

- Функциональные требования — это то, что система должна делать. У твоего проекта следующие возможности:
- Создание событий через голос или текст на русском и английском языках
- Редактирование и удаление существующих событий
- Синхронизация с Google Calendar и Apple Calendar
- Автоматический поиск оптимального времени для события в календаре
- Воспроизведение озвучивания подтверждения события
- Система напоминаний (встроенные напоминания календарей + опция выбора времени системой)
- Регистрация/авторизация пользователей
- OAuth 2.0 интеграция для подключения Google Calendar и Apple Calendar
- Логирование всех запросов и действий для последующего анализа
- Сохранение аудиофайлов для исправления ошибок STT

Нефункциональные требования

- Нефункциональные требования — это как система должна работать:
- **Производительность:** API должен ответить в течение 2 секунд, обработка голоса — до 3 секунд
- **Доступность:** Система должна работать 99% времени (допускаются короткие перерывы на обновления)
- **Масштабируемость:** Должна выдержать 1000 активных пользователей без проблем
- **Безопасность:** Все токены хранятся в зашифрованном виде, все коммуникации по HTTPS
- **Совместимость:** Поддержка разных форматов аудио (WAV, MP3, OGG), работа с современными браузерами

1.2 Расчет нагрузки на систему

Давайте посчитаем, какую нагрузку должна выдержать система на основе следующих параметров:

Параметр	Значение
Активные пользователи в день	1000
Событий на пользователя в день	5-10 (среднее 7)
Всего операций в день	7000 запросов
RPS (requests per second)	0.08 в среднем, до 0.56 в пик

Размеры данных

- **Текстовый запрос:** 200-500 байт
- **Голосовой запрос (аудиофайл):** 500 KB - 2 MB за минуту
- **История одного пользователя за год:** ~5 MB
- **Всего на 1000 пользователей в год:** ~5 GB

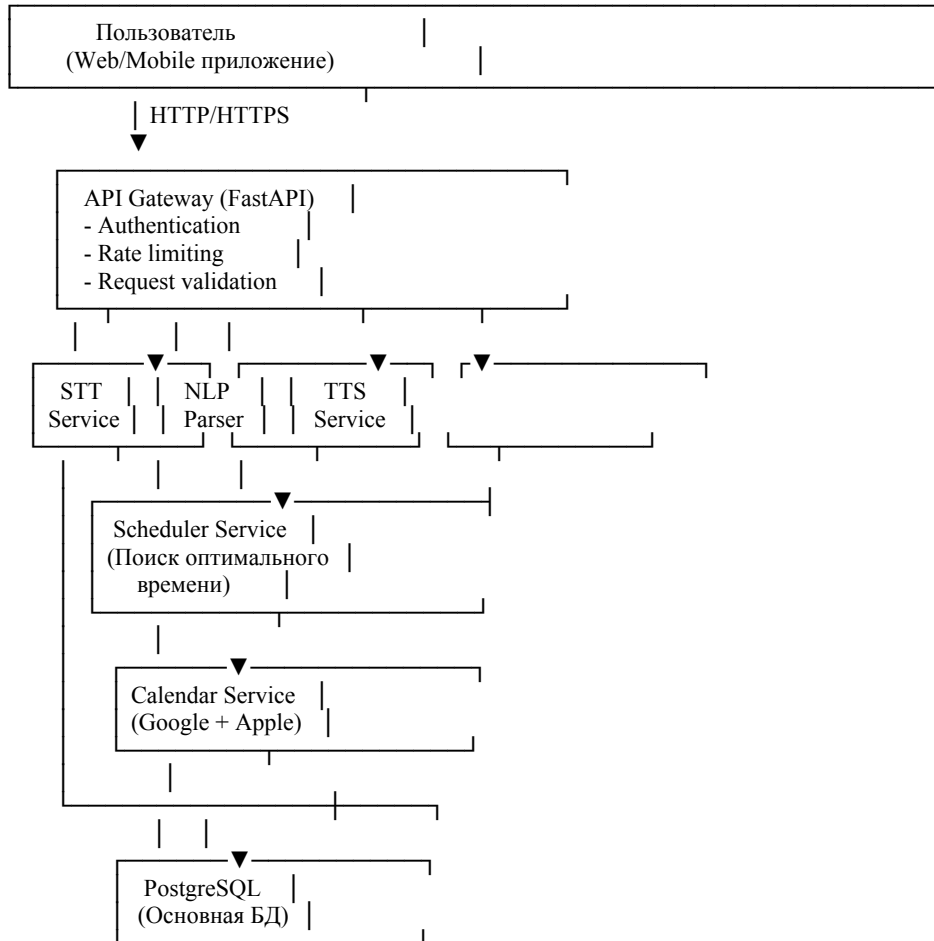
Требования к ресурсам

- **CPU:** 1 ядро будет достаточно
- **RAM:** 2-4 GB (для кеша и обработки)
- **Диск:** 50-100 GB (с учетом аудиофайлов)
- **Пропускная способность:** Минимальная

1.3 Высокоуровневый дизайн

Представим систему как набор компонентов, которые взаимодействуют между собой.

Архитектурная схема



Компоненты системы

- **API Gateway (FastAPI):** Главный вход, обрабатывает все запросы, проверяет права доступа
- **STT Service:** Преобразование аудиофайла в текст с помощью Whisper API
- **NLP Parser:** Разбор текста на части (дата, время, описание события)
- **Scheduler Service:** Поиск свободных слотов и выбор оптимального времени
- **Calendar Service:** Общение с Google Calendar и Apple Calendar API
- **TTS Service:** Преобразование текста в звук для подтверждения
- **PostgreSQL:** Хранение пользователей, событий, логов и аудиофайлов
- **Redis:** (опционально) Кеширование и хранение сессий

Поток данных

1. Пользователь отправляет голос или текст в API Gateway
2. API Gateway проверяет аутентификацию, валидирует данные
3. Если это голос → STT Service преобразует в текст
4. NLP Parser разбирает текст, извлекает информацию
5. Scheduler Service смотрит в календарь, находит оптимальное время
6. Calendar Service создает событие в Google/Apple Calendar
7. TTS Service озвучивает подтверждение
8. Все логируется в PostgreSQL, результат возвращается пользователю

1.4 Выбор базы данных

Тебе нужно хранить разные типы данных, поэтому комбинация баз имеет смысл.

PostgreSQL — основная база данных

Реляционная БД, идеально подходит для структурированных данных. Основные таблицы:

- **users:** user_id, email, password_hash, created_at, language
- **calendar_tokens:** token_id, user_id, calendar_type, access_token, refresh_token, expires_at
- **events:** event_id, user_id, title, description, start_time, duration, event_type, calendar_type, calendar_event_id, created_at, updated_at, is_deleted
- **reminders:** reminder_id, event_id, reminder_time, reminder_type, sent_at
- **request_logs:** log_id, user_id, request_type, input_text, output_event_id, processing_time, status, created_at
- **audio_files:** file_id, user_id, file_path, transcribed_text, accuracy, uploaded_at

Преимущества PostgreSQL:

- Надежная, используется везде в бизнесе
- ACID гарантии (если начала писать в БД, то напишется полностью или не напишется вообще)
- Хорошо работает с JSON данными
- Легко масштабировать потом если понадобится

Redis — кеш и сессии (опционально)

Очень быстрая в памяти база данных. Используй для:

- Активные сессии пользователей (JWT токены)
- Кеш свободных слотов календаря (чтобы не дергать API каждый раз)
- Rate limiting (понимать, сколько запросов сделал пользователь)

S3 или MinIO — хранилище файлов (опционально)

Для аудиофайлов. На начальном этапе подойдет обычная папка на диске, потом перейдешь на S3.

1.5 Низкоуровневый дизайн отдельных сервисов

STT Service (Speech-to-Text)

Это самая простая часть. Берешь аудиофайл, отправляешь в Whisper:

- Загрузить аудио файл (WAV, MP3)
- Отправить в Whisper API
- Получить текст обратно
- Сохранить аудиофайл в БД для анализа
- Вернуть текст на русском или английском

Проблемы и решения:

- **Большой аудиофайл:** используй async, чтобы не блокировать другие запросы
- **Whisper недоступен:** сохрани ошибку в логи, предложи пользователю повторить
- **Текст неправильный:** сохрани аудиофайл, потом проанализируешь ошибку

NLP Parser Service

Разбор текста на части:

- Вход: "Встреча с командой завтра в 15:00"
 - Дата: "завтра" → 2025-11-08
 - Время: "15:00" → 15:00
 - Тип события: "встреча" → meeting
 - Описание: "с командой"

Сейчас используешь регулярные выражения. Позже можешь заменить на нейросеть, но для студенческого проекта регулярных выражений достаточно.

Scheduler Service (поиск оптимального времени)

Это самая интересная и сложная часть. Алгоритм работает так:

- Получить все события на день из календаря
- Найти все свободные окна (gaps)
- Оценить каждое окно по критериям:
 - Логичность: обед 12-14, тренировка после работы, встреча днем
 - Продолжительность: окно $\geq$ длительности события
 - Удобство: не в 6 утра и не в 23:00

- Предпочтения пользователя
- Выбрать окно с наибольшей оценкой

Calendar Integration Service

Общение с внешними API (Google Calendar и Apple Calendar).

Google Calendar:

- OAuth 2.0 для получения доступа
- API URL: <https://www.googleapis.com/calendar/v3/>
- Основные операции: GET (получить события), POST (создать), PATCH (отредактировать), DELETE (удалить)

Apple Calendar:

- CalDAV протокол или REST API
- Принцип похожий на Google, но реализация немного отличается

Обработка ошибок:

- Retry с exponential backoff: $1s \rightarrow 2s \rightarrow 4s$
- Circuit breaker pattern: если API часто падает, перестать ему звонить
- Graceful degradation: если не можешь создать в Google, предложи сохранить локально

TTS Service (Text-to-Speech)

Озвучивание текста. Сейчас используешь gTTS, это нормально.

Можешь улучшить:

- Сохранять популярные фразы в кеш (не генерировать каждый раз одно и то же)
- Использовать более качественные TTS (например, Google Cloud Text-to-Speech API)

1.6 Архитектура: монолит или микросервисы?

Рекомендуно монолит с хорошей структурой кода.

Почему монолит?

- Студенческий проект, нет смысла усложнять
- Легче разворачивать и поддерживать
- Проще отлаживать проблемы

Структура проекта

```
ai-planner/
├── app/
│   ├── main.py      # FastAPI приложение
│   └── api/
│       ├── auth.py   # Регистрация, авторизация
│       ├── events.py  # Создание, редактирование, удаление
│       ├── schedule.py # Основной endpoint
│       └── calendar.py # Управление календарем
│   └── services/
│       ├── stt.py     # STT Service
│       ├── nlp.py     # NLP Parser
│       ├── scheduler.py # Поиск оптимального времени
│       ├── calendar.py # Интеграция с Google/Apple
│       └── tts.py     # TTS Service
│   └── models/
│       ├── user.py    # Модель пользователя
│       ├── event.py   # Модель события
│       └── schemas.py  # Pydantic schemas
│   └── database/
│       ├── db.py      # Подключение к БД
│       └── models.py   # ORM модели
│   └── utils/
│       ├── auth.py    # Хелперы аутентификации
│       ├── logger.py  # Логирование
│       └── config.py   # Конфигурация
├── tests/
├── docker-compose.yml
├── Dockerfile
└── requirements.txt
```


1.7 Deployment: Docker и Docker Compose

Для студенческого проекта лучший вариант — Docker Compose. Это позволяет:

- Быстро развернуть на любой машине
- Запустить PostgreSQL и Redis одной командой
- Все конфигурируется в одном файле

docker-compose.yml должен содержать:

```
version: '3.8'

services:
  api:
    build: .
    ports:
      - "8000:8000"
    environment:
      - DATABASE_URL=postgresql://user:pass@db:5432/ai_planner
      - REDIS_URL=redis://redis:6379
    depends_on:
      - db
      - redis
    command: uvicorn app.main:app --host 0.0.0.0 --port 8000

  db:
    image: postgres:15
    environment:
      - POSTGRES_USER=user
      - POSTGRES_PASSWORD=password
      - POSTGRES_DB=ai_planner
    volumes:
      - postgres_data:/var/lib/postgresql/data

  redis:
    image: redis:7-alpine
    volumes:
      - redis_data:/data

volumes:
  postgres_data:
  redis_data:
```

1.8 CI/CD Pipeline

На начальном этапе минимум, который стоит сделать:

- Pre-commit hooks: проверка кода перед коммитом (linting, formatting)
- GitHub Actions: автоматические проверки при push

GitHub Actions workflow запускает:

- Все тесты
- Проверку кода на ошибки
- Сборку Docker образа

Резюме: полный системный дизайн

Компонент	Описание
1. Функциональные требования	Создание/редактирование/удаление событий, синхронизация с календарями, голосовой ввод, многоязычность, напоминания
2. Нагрузка	7000 запросов в день, пиковая 0.56 RPS — один сервер легко выдержит
3. Архитектура	Единый FastAPI сервер с разделением на компоненты (STT, NLP, Scheduler, Calendar, TTS)
4. База данных	PostgreSQL для основных данных, Redis для кеша, файловое хранилище для аудио
5. Развертывание	Docker Compose для разработки и production
6. Структура	Монолит с хорошей организацией кода для возможности перехода на микросервисы
7. CI/CD	Pre-commit hooks и GitHub Actions для автоматизации проверок